

## **Cloud Computing Technology – Concise Summary**

### **Clients in Cloud Computing**

- Clients are the devices end users use to access cloud services.
  - Different client types offer different ways to interact with cloud data and applications, depending on organizational needs.
- 

### **Mobile Clients**

- Include laptops, PDAs, and smartphones.
  - Laptops can use cloud applications fully; smartphones/PDAs have limited capability.
  - Challenges: variable speed, weaker security, and risk of device loss.
  - Benefit: If data is stored on the cloud instead of the device, stolen devices expose minimal data.
- 

### **Thin Clients**

- Devices with no hard drives; simply display what the server provides.
  - Ideal for in-house clouds or virtualized environments.
  - Benefits: low cost, low maintenance, low energy use, high security (no data stored locally).
- 

### **Thick Clients**

- Traditional computers with applications installed locally.
  - Necessary when certain applications must remain in-house.
  - Can still connect to virtualized servers.
  - Weaknesses: more vulnerable to data theft, local failures result in data loss and reconfiguration.
- 

## **Cloud Security**

### **General Security Concerns**

- Biggest challenge: data stored by a third party.
  - Benefits exist too: centralized data reduces leakage, cloud providers often offer stronger security.
- 

### **Data Leakage**

- Centralized storage reduces risks from laptops and desktops storing sensitive data.
  - Thin clients especially help minimize leakage.
- 

### **Offloading Work to Providers**

- Providers handle delegation, outsourcing, automation, and collaboration.
  - Cloud vendors can invest in stronger security due to economies of scale and reputation concerns.
- 

### **Logging**

- Logging improves in the cloud because providers can allocate unlimited virtual resources.
  - Types: system logging, application logging, security logging, audit logging.
- 

### **Forensics**

- Cloud providers can respond to incidents faster and with less downtime.
  - Virtual machines can be cloned for offline analysis.
  - Forensic types: digital, DNA, fingerprint, cyber, firearms.
  - Many organizations lack internal incident response teams; cloud simplifies this.
- 

### **Security Product Development**

- Security vendors are actively creating solutions for cloud and virtual machines.
  - The cloud offers new opportunities for improved security tools.
- 

### **Auditing**

- Cloud raises new auditing challenges because data is stored on someone else's equipment.
  - Types: financial, compliance, operational, IT, internal audits.
- 

### **Compliance**

- SaaS providers face the same issues as organizations: securing hardware, network, applications, and data.
- Regulations like SOX, GLBA, HIPAA, and PCI DSS complicate cloud compliance.
- Harder to identify where sensitive data is stored because SaaS may use multiple partner systems.

---

### **Web Application Breaches**

- Providers must follow OWASP security guidelines (required in PCI 6.5).
- Customers should verify how providers handle breaches and where data is stored.
- Cloud servers may be located globally (e.g., Brazil, Germany, Thailand).

---

### **VPNs in Cloud Computing**

#### **Remote Access**

- Cloud makes telecommuting easier, reducing office space and utility needs.
- Organizations still need secure remote access whether users are in-office or remote.

---

#### **SSL VPN**

- SSL VPN uses standard web browsers and requires no special client software.
- More user-friendly compared to IPsec VPNs.
- Based on SSL encryption using RSA public/private keys.

#### **Advantages of SSL VPN**

- Secures data transmission without heavy PKI overhead.
- Lightweight client management; easy for end users.
- Enforces endpoint security by checking:
  - Antivirus status
  - OS patch level
  - Malware/bot presence
- Provides simple, inexpensive, and efficient access security.
- Supports telecommuting securely.

### **Key Management in Cloud Computing – Concise Summary**

- Storing data off-site introduces risks because applications, compute cycles, and storage are not under your direct control.
- Data may be exposed due to accidents, improper deletion, or providers not erasing freed/failed storage.
- Man-in-the-middle attacks are also a concern.

- To mitigate risks, remote services and servers must be **cryptographically authenticated** using client/server certificates.
  - Authorization systems like **Kerberos** ensure proper authentication.
  - Cloud storage must be **encrypted**, and data should automatically become inaccessible when its **encryption key is destroyed**.
  - This requires managing many keys, including:
    - Transport keys
    - Authentication keys
    - Authorization tokens
    - File encryption keys
    - Hardware storage keys
    - Revocation keys
    - Certificates
- 

### **Basic Public Internet (Cloud Connectivity Option)**

#### **Description:**

- The simplest and most common connection method, using broadband or dial-up Internet.
- No special optimizations (e.g., TCP acceleration or compression).

#### **Advantages:**

- Accessible to anyone with Internet access.
- High fault tolerance.
- Many provider options.
- SSL/HTTPS encryption provides confidentiality.
- Cost-effective.

#### **Disadvantages:**

- No end-to-end Quality of Service (QoS) → difficult to meet SLAs.
  - Poor performance over high-latency connections (TCP/HTTP inefficiencies).
  - Downtime may be outside your control.
- 

### **Accelerated Internet**

- Adds advanced delivery features on top of a basic Internet connection.

- Can increase cloud performance by **20%–50%** by offloading network tasks like SSL termination and TCP management.
- Caching, compression, and prefetching can improve user performance by **50% or more**.
- Used by providers like AT&T Hosting, Citrix NetScaler, and F5 Web Accelerator.

**Notes:**

- Customers should focus on SLAs and bandwidth charges rather than specific optimization methods.
  - Requires installation of a **server-side appliance** and usually a **client-side downloadable agent**.
- 

**Optimized Internet Overlay**

- Uses the public Internet, but optimization occurs inside the provider's cloud network at **Points of Presence (POPs)**.

**Enhancements:**

- Real-time optimized routing → avoids congestion and improves SLA compliance.
- SSL sessions may be temporarily terminated to optimize payloads and then re-encrypted.
- Some application logic runs at the POP → better scalability, fault tolerance, and **up to 80% improved response time**.
- Frequently accessed content delivered from local caches.

**Disadvantages:**

- More expensive—up to **4× the cost** of basic Internet connectivity.
- Strong **vendor lock-in**, because application components run inside the carrier's network.

**Site-to-Site VPN (Private WAN / MPLS)**

- Uses a private WAN (typically MPLS/VPN) to connect directly to the service provider.
  - Provides confidentiality, guaranteed bandwidth, and SLAs covering availability, latency, and packet loss.
  - MPLS can scale bandwidth as needed and supports QoS in SLAs.
  - However, private WANs are not always more reliable than Internet connections, even with redundant ISP links.
- 

**Cloud Providers**

- Providers with distributed services need robust connections; private tunnels ensure stable bandwidth, low latency, and low packet loss.
  - Encryption and strong authentication add security.
  - As providers grow, bandwidth costs rise due to heavy client traffic and cross-site traffic.
  - Major providers (e.g., Google) avoid these costs by building their own WANs and establishing multiple peering points with ISPs.
- 

### Cloud Consumers

- Large enterprises may build private distributed infrastructures with datacenters connected via private fiber.
  - Viability depends on distance, bandwidth needs, and budget.
  - Major offices access cloud apps through the corporate WAN.
  - Smaller offices and mobile users rely on VPN over optimized or accelerated Internet.
  - VPN tunnels over the public Internet are suitable only when high performance is not essential.
- 

### Pipe Size (Bandwidth Requirements)

- Bandwidth = transmission speed / throughput.
  - Three uncontrollable factors restrict bandwidth needs:
    - Internet bandwidth between the organization and the cloud
    - Round-trip time (RTT)
    - Cloud response time
- 

### Upstream vs. Downstream (Symmetric vs. Asymmetric Links)

- **Symmetric connection:** Upload and download speeds are equal.
- **Asymmetric connection:** Upload speed is lower than download speed (e.g., ADSL: 1.5 Mbps down / 750 Kbps up).
- Organizations usually use symmetric, business-grade connections.
- Internet performance varies constantly due to routing and network changes.
- Occasional slowdowns are normal; only consistent problems indicate issues.
- A good benchmark: if you achieve **85% of your rated bandwidth consistently**, your connection is performing well.
- Test Internet speeds multiple times a day, especially in the morning.

## How Much Bandwidth Do We Need?

- Estimate how much data will move in and out of the cloud to determine the required connection size (“pipe”).
  - Most organizations already have a strong enough Internet connection for cloud use.
  - As more services move to the cloud, bandwidth demand increases—insufficient capacity leads to slowdowns.
  - Calculate expected usage and ensure adequate resources are available.
  - Secure an SLA that guarantees required bandwidth; SLAs also provide remedies if providers fail to meet performance levels.
- 

## Redundancy

- Reliability and uptime must be planned with redundancy and failover in mind.
  - Traditional redundancy = extra servers in the data center.
  - Modern redundancy (with virtualization) = virtual servers cloned on the same or different physical machines.
  - In the cloud, data and workloads are scattered across multiple locations, making redundancy more complex.
  - Providers create redundant systems by reallocating cloud resources, not by adding physical servers.
- 

## Cloud Services and Identity

### Identity Services

- Applications—whether in-house or cloud-based—must identify users using a **digital identity**.
- In-house identity is usually managed by **Active Directory**.
- Cloud platforms use their own identity systems:
  - Amazon → Amazon identity
  - Google App Engine → Google account
  - Microsoft cloud apps → Windows Live ID
- Identity systems influence how cloud infrastructure is deployed.

### OpenID

- OpenID is an open, decentralized single sign-on system allowing one digital identity for multiple services.
- Identity is represented as a **URL**, and no central authority is required for authentication.

- Supports different authentication types (smart cards, biometrics, passwords).
- Widely used by:
  - Google
  - IBM
  - Microsoft
  - Yahoo

## Integration

- Applications increasingly need to communicate with each other, traditionally handled through on-premises tools like message queues and integration servers.
  - Cloud platforms now offer integration services as well.
  - **Amazon SQS** allows applications to exchange messages through cloud queues.
    - Messages are replicated across multiple queues, so an application may not see every message on each request.
    - Delivery order is **not guaranteed**.
    - These simplifications improve scalability but require developers to design differently compared to on-premises systems.
  - **BizTalk Services** uses a relay service instead of queues, enabling communication through firewalls.
  - Cloud integration must handle communication across multiple organizations and overcome firewall barriers.
- 

## Mapping

- Maps are widely used in web applications (e.g., hotel/restaurant sites showing locations and directions).
  - Building mapping databases is expensive, so cloud solutions provide this functionality.
  - **Google Maps** and **Microsoft Virtual Earth** let developers embed maps in websites and applications.
- 

## Payments

- Organizations may choose to accept online payments.
- Cloud payment services make this simple:

- Credit card processing services
    - PayPal and similar platforms
  - Customers can send money directly to the organization via these cloud-based services.
- 

## Search

- Websites and applications often embed search features.
- **Microsoft Live Search** supports both on-site and cloud applications by returning search results.
- Searches can target:
  - Internal organizational databases
  - The public Internet
- Example: a movie database app can return results from its own database plus results from the whole web.
- Basic cloud access only requires a computer and an Internet connection.

## Web Application Framework

- A web application framework helps develop dynamic websites, web applications, and web services.
  - Frameworks reduce development effort by handling common tasks.
  - They provide pre-written libraries so developers don't need to write the same code repeatedly.
  - Early web pages were static HTML and had to be manually updated by the author.
  - The introduction of **CGI (Common Gateway Interface)** allowed external programs to interact with web servers, making websites more dynamic.
- 

## AJAX (Asynchronous JavaScript and XML)

- AJAX is a set of techniques for building interactive web applications.
- It allows web pages to retrieve data from the server **asynchronously**, without reloading the page.
- Server communication happens in the background and does not interrupt the current page.

## AJAX Technologies

- XHTML and CSS for presentation.
- Document Object Model (DOM) for dynamic page interaction.
- XML and XSLT for data exchange and transformation.
- XMLHttpRequest object for asynchronous communication.
- JavaScript integrates all these technologies.
- AJAX is evolving and can also use **JSON**, HTML, or plain text instead of XML.

#### **Advantages of AJAX**

- Reduces repeated content across multiple pages.
- Fetches only required data, saving bandwidth and reducing load time.
- Improves responsiveness and user interaction.
- Scripts and style sheets are downloaded once, reducing server connections.

#### **Disadvantages of AJAX**

- Dynamic pages do not work well with browser Back/Forward buttons.
  - Bookmarking dynamic pages is difficult.
  - Requires browser support and JavaScript enabled.
  - No single standards body defines best practices for AJAX testing.
- 

#### **Python Django Framework**

- Django is an open-source web framework written in Python.
- Initially built for news websites and released publicly in 2005 under a BSD license.
- Managed by the **Django Software Foundation** since 2008.
- Designed for database-driven websites with reusable components.
- Uses the **DRY (Don't Repeat Yourself)** principle.
- Provides an automatically generated admin interface for CRUD operations.

#### **Django Core Features**

- Built-in lightweight development web server.
  - Caching framework supporting multiple cache methods.
  - Internal signaling system for component communication.
  - Internationalization support for multiple languages.
  - Extendable template engine.
-

## Web Hosting Services

- Web hosting services store applications and data in the cloud.
  - Examples include **Amazon EC2** and **Mosso**.
- 

### Amazon Elastic Compute Cloud (EC2)

- EC2 provides scalable computing resources in the cloud.
- Allows users to configure and launch servers quickly.
- Gives full control over virtual computing environments.
- Server instances can be started in minutes and scaled up or down easily.
- Users pay only for the computing resources they use.

### EC2 Architecture

- Uses **Xen virtualization**.
- Virtual machines are called **instances**.
- Instance sizes: small, large, and extra large.
- Performance measured in **EC2 Compute Units**.
- One EC2 Compute Unit  $\approx$  1.0–1.2 GHz 2007 Opteron or Xeon processor.
- Initially supported Solaris; later added Linux and Windows Server 2003.

### Mosso

- Mosso provides **enterprise-grade cloud hosting and storage** through *The Hosting Cloud* and *CloudFS*.
- Offers an easy-to-manage interface for developers, designers, and IT managers to quickly deploy reliable web applications and high-performance storage.

### Mosso Components

- **Cloud Sites**
  - Runs Windows or Linux applications across hundreds of servers.
  - Marketed as a fast way to deploy websites in the cloud.
- **Cloud Files**
  - Provides unlimited online storage for media (backups, videos, user content).
  - Content is delivered through Limelight Networks' CDN.
- **Cloud Servers**

- Allows instant deployment of one to hundreds of cloud servers.
  - Supports advanced, high-availability architectures.
- 

### Proprietary Cloud Methods

- Some vendors build their own cloud infrastructures.
  - **Microsoft Azure** and **Force.com** are examples of proprietary cloud platforms.
- 

### Microsoft Azure

- Azure Services Platform is Microsoft's cloud solution spanning cloud and enterprise datacenters.
- Delivers applications and services across PCs, web, and mobile devices.
- Combines developer tools with cloud-based **storage, compute, and networking** services.
- Supports deployment both in the cloud and on-premises.
- Allows developers to use existing skills and tools such as **.NET Framework** and **Visual Studio**.
- Supports open and commercial tools and standards like **HTTP, REST, and AtomPub**.

### Key Azure Components

- **Windows Azure** – hosting, management, scalable storage, compute, and networking.
  - **Microsoft SQL Services** – database and reporting services.
  - **Microsoft .NET Services** – workflow and access control services.
  - **Live Services** – storage, sharing, and synchronization across devices.
  - **SharePoint Services & Dynamics CRM Services** – collaboration and business solutions.
  - Microsoft operates global datacenters (e.g., Washington, Texas, with others planned in Chicago and Dublin).
- 

### Force.com (Salesforce.com)

- A **Platform as a Service (PaaS)** for building and deploying business applications.
- Simplifies development by replacing complex software platforms with a scalable service.
- Allows multiple applications to run in a single Salesforce instance.
- Applications share a common **security model, data model, and UI**.
- Provides an on-demand operating system-like environment.

### Force.com Features

- On-demand database creation.

- Workflow engine for collaboration.
  - **Apex Code** language for complex business logic.
  - Web Services APIs for integration and mashups.
  - **Visualforce** framework for building user interfaces.
- 

### Visualforce

- Enables creation of user interfaces for any device or screen.
- Uses **HTML, AJAX, and Flex**.
- Works with Apex Code for logic and workflow intelligence.
- Provides page-based UI development with reusable components and controllers.

### Visualforce Capabilities

- **Pages** – define UI using standard web technologies.
- **Components** – reusable UI elements matching Salesforce look and feel.
- **Controllers** – define UI behavior and interaction logic.
- Supports reuse of standard Salesforce actions (new, edit, save).

### Web Sample Applications – Google Apps

- Different cloud providers offer different services; Google is a major cloud provider with strong enterprise offerings.
- **Google Apps**, launched in August 2006, is a cloud-based application suite designed for businesses and organizations.

### Core Google Apps Services

- **Gmail** – web-based email service
  - **Google Calendar** – shared calendaring
  - **Google Talk** – instant messaging and VoIP
  - **Start Page** – customizable home page for a domain
  - Used by **100,000+ small businesses** and **hundreds of universities**.
  - Enterprises value Google Apps for easy access, collaboration, and effective communication.
  - Google also offers a paid version called **Google Apps Premier Edition**.
- 

### Google Apps Premier Edition – Key Features

- **10 GB storage per user**, greatly reducing the need to delete email.

- **Business APIs** for data migration, user provisioning, single sign-on, and mail gateway integration.
  - **99.9% uptime SLA** for Gmail, with monitoring and service credits if targets are not met.
  - **24/7 support** for critical issues, including phone support for administrators.
  - **Optional advertising**, turned off by default.
  - **Low cost**: US\$50 per user per year.
- 

#### **Additional Google Apps Features (All Editions)**

- **Google Docs and Spreadsheets** for real-time collaboration.
  - Multiple users can work on documents simultaneously.
  - All document changes are tracked, with administrative controls for sharing limits.
- 

#### **Mobile and Administration Features**

- **Gmail for mobile devices** (including BlackBerry) with full Gmail features like search and conversation view.
- Supports calendar sync tools and Google Talk clients for mobile users.
- **Application-level controls** let administrators enforce business policies.
- **Enterprise partners** (e.g., Avaya, Postini) offer enhanced security, email gateways, calendar sync, and third-party integrations.
- Google-hosted applications support multiple languages, including European and Asian languages.

#### **APIs and Cloud Access**

##### **What Are APIs?**

- An **API (Application Programming Interface)** is a set of rules and instructions that lets software programs communicate with each other.
- Companies publish APIs so developers can build applications using their services.
- Example: Amazon's API allows third-party websites to link directly to Amazon products.
- APIs enable **program-to-program communication** without user involvement.
- Behind-the-scenes tasks (e.g., credit card verification on Amazon) are handled through APIs.
- APIs are similar to SaaS because developers reuse existing services instead of building everything from scratch.

---

## How APIs Work

- APIs define how two systems communicate using **web services**.
- Web services rely on standards like **XML**, which structures data for human and machine readability.
- APIs exchange XML-based messages (example: Google Maps API).
- Developers use APIs to automate tasks such as archiving emails to the cloud.
- APIs are usually distributed as part of an **SDK** with tools and documentation.
- APIs run invisibly in the background for users.

## API-Related Standards

- **SOAP** – Encodes XML messages for transmission across networks.
- **UDDI** – Directory for discovering and listing web services.
- **WSDL** – Describes services listed in UDDI and how to interact with them.

---

## API Creators

- Organizations may use existing APIs or create their own to integrate cloud applications.

---

## Google APIs and Gadgets

### Google Gadgets

- Google Gadgets allow desktop and web-based searching through **Google Desktop Search**.
- Built using three technologies:
  - **XML** – Defines gadget structure and data sources.
  - **HTML** – Handles page layout and static content.
  - **JavaScript** – Adds dynamic behavior.

### Google Desktop Search Features

- Searches emails, files (Word, Excel, PowerPoint), browser history, and chat logs.
  - Very fast searching using Google's search technology.
  - Searches both the Web and local computer simultaneously.
  - Results from the local computer appear alongside Google.com results.
  - Continuously updates search index for near-instant results.
-

## Google Data APIs

- Provide a standard way to read and write data on the Web.
- REST-based APIs using **AtomPub**, **HTTP**, and **Atom syndication format**.
- Used to integrate cloud and enterprise applications.

## Examples of Google Data APIs

- Google Apps, Calendar, Contacts, Docs, Spreadsheets
  - Blogger, YouTube, Picasa
  - Google Finance, Health, Books, Webmaster Tools, Code Search
- 

## Salesforce.com and Google Partnership

- Salesforce partnered with Google to simplify cloud application development.
  - **Force.com Toolkit for Google Data APIs** enables integration between Salesforce apps and Google services.
  - Developers get a **multi-cloud platform** for building web applications.
  - Allows extending Salesforce for Google Apps or building new cloud-based business applications.
- 

## GoGrid API

- GoGrid API enables programmatic control of cloud hosting infrastructure.
  - Supports:
    - Auto-scaling servers
    - Managing IP addresses
    - Creating/deleting servers
    - Viewing billing details
  - REST-like API accessible over the Internet.
  - Requires GoGrid account and programming knowledge.
  - Supports Java, PHP, Python, and Ruby.
- 

## Apex (Salesforce Platform)

### Apex Web Services API

- Handles over **50% of Salesforce.com's service transactions**.
- Enables complex data retrieval (similar to database JOINS) in a single request.

- Improves performance and simplifies integrations.

### Apex Development Platform

- Used to build SaaS applications on Salesforce CRM.
- Provides access to Salesforce's backend database and services.
- Eliminates the need to build core SaaS infrastructure.

### Apex Platform Tools

- **Apex Builder** – Drag-and-drop customization.
- **Apex API** – Access Salesforce data from external programs.
- **Apex Code** – Server-side programming language that reduces client-server calls.
- Sample Apex code demonstrates preventing duplicate records using email address validation.

### Cloud Storage – Overview

- Cloud storage lets users access data from **any location with Internet access**.
- Users do not need the same computer or physical storage devices.
- Cloud storage systems vary widely:
  - Some store specific data (email, photos, videos)
  - Others store any type of digital data
- Providers range from small services to warehouse-scale systems.
- A basic cloud storage setup involves:
  - A data server connected to the Internet
  - Users upload data via the Internet
  - Data is accessed through a web-based interface for download or manipulation

---

### Redundancy and Availability

- Cloud storage uses **dozens or hundreds of servers**.
- Data is replicated across multiple machines for redundancy.
- Copies may be stored using **different power supplies** to ensure availability.
- Redundancy ensures users can access data even during failures.
- Many users adopt cloud storage primarily for **data safety and disaster recovery**.

---

## Storage as a Service (StaaS)

- Storage as a Service means renting storage space from a third-party provider.
  - Ideal for organizations lacking budget or technical expertise.
  - Popular due to complex backup, replication, and disaster recovery needs.
  - Users pay:
    - Per gigabyte stored
    - Or per amount of data transferred
  - No need to buy or maintain infrastructure.
  - Client software defines backup sets and transfers data over a WAN.
  - Data can be restored from the provider after data loss.
- 

## Cloud Storage Providers

- Hundreds of providers exist, including specialized services:
    - **Google Docs** – documents, spreadsheets, presentations
    - **Email providers** (Gmail, Hotmail, Yahoo!) – email storage
    - **Flickr, Picasa** – photo storage
    - **YouTube** – video storage
    - **Hostmonster, GoDaddy** – website data
    - **Facebook, MySpace** – user content
    - **MediaMax, Strongspace** – general-purpose storage
  - Some services are free; others charge based on storage and data transfer.
  - Competition has reduced prices, and many providers offer free tiers.
- 

## Security Concerns

- **Encryption** protects stored data using complex algorithms and keys.
- **Authentication** uses usernames and passwords.
- **Authorization** controls who can access which data.
- Organizations may define multiple access levels.
- Risks still exist:
  - Hackers gaining access

- Insider threats from disgruntled employees
- 

### Reliability

- Reliability is critical; unstable providers become liabilities.
  - Providers use redundancy to improve uptime, but failures can still occur.
  - Reputation is essential for providers in a competitive market.
  - Providers perceived as unreliable quickly lose customers.
- 

### Advantages of Cloud Storage

- Data is distributed across datacenters, not tied to a single location.
  - Providers balance loads and move data closer to users for faster access.
  - Excellent for **disaster recovery**:
    - Off-site data survives fires, floods, or hurricanes
  - Can be the difference between business recovery and permanent shutdown.
- 

### Vendor Selection Considerations

- Choosing a vendor can be complex.
  - Storage methods vary:
    - Agent-based automatic transfers (e.g., FTP)
    - Web-based manual uploads
  - **Amazon S3** is widely known, but enterprises may prefer providers with:
    - SLAs
    - Direct customer support
- 

### Cautions and Risks

- Cloud storage is still evolving; a **hybrid approach** is recommended.
- Avoid moving all critical data immediately.
- Large enterprises may face:
  - Application rewrites
  - Lack of portability
- Some vendors (e.g., 3tera) support multiple platforms (LAMP, Java, .NET).

- Key concerns when choosing a provider:
    - Price
    - Reliability
  - Many companies test services gradually before full adoption.
- 

### Legal and Compliance Issues

- Copyrighted content (music, video) may have licensing restrictions.
  - Vendors provide different guarantees regarding data loss or compromise.
  - Best practice: maintain **multiple backups**:
    - Local
    - Off-site
    - Sync and archive systems
- 

### Outages

- Cloud outages can disrupt access:
    - Example: Amazon S3 outage (Feb 2008)
  - No data was lost due to multiple replicas.
  - However, applications went offline temporarily.
  - Highlights the importance of cautious cloud adoption.
- 

### Data Theft Risks

- Data stored outside the organization increases security exposure.
  - Multi-tenant environments store data alongside other companies'.
  - Strong encryption and secure transit (e.g., **SSL**) are essential.
- 

### Major Cloud Storage Providers

- Many providers exist; major players include:
  - **Amazon**
  - **Nirvanix**
  - **Google** (GDrive – planned)
  - **EMC**

- IBM Blue Cloud
- Provider comparison is necessary to match services with business needs.

### Amazon Simple Storage Service (S3)

- **Amazon S3**, launched in 2006, is the most well-known cloud storage service.
- Designed to make **web-scale storage easy** for developers.
- Provides a **simple web services interface** to store and retrieve any amount of data from anywhere on the Web.
- Uses the same **scalable infrastructure** Amazon relies on for its own global websites.
- Focuses on leveraging economies of scale to reduce costs for developers.

### Core Features

- Store objects from **1 byte to 5 GB**.
  - **Unlimited number of objects**.
  - Each object accessed using a **unique key**.
  - Objects can be **private or public**, with user-specific permissions.
  - Supports **REST and SOAP APIs**, compatible with most development toolkits.
- 

### S3 Design Requirements

- **Scalable**: Supports unlimited storage, users, and request rates.
  - **Reliable**: Designed for **99.99% availability** with no expected downtime.
  - **Fast**: Server latency kept lower than Internet latency; performance scales by adding nodes.
  - **Inexpensive**: Built on commodity hardware; failures are expected and tolerated.
  - **Simple**: Easy to use while still being highly scalable and reliable.
  - One S3 system supports both **Amazon's internal applications** and **external developers**.
- 

### S3 Design Principles

- **Decentralization** to remove bottlenecks and single points of failure.
- **Autonomy**: Components make local decisions.
- **Local responsibility** for consistency.
- **Controlled concurrency** and **parallelism**.
- **Failure tolerance** as a normal operating condition.

- **Symmetry:** All nodes have identical roles.
  - Built from **small, simple building blocks**.
- 

### How Amazon S3 Works

- Stores data as **objects** ( $\leq 5$  GB) with up to **2 KB of metadata**.
  - Objects are stored in **buckets**, owned by AWS accounts.
  - Access via **REST, SOAP, HTTP GET, or BitTorrent**.
  - Uses **Access Control Lists (ACLs)** for permissions.
  - Objects can be accessed through URLs:
    - `http://s3.amazonaws.com/bucket/key`
    - `http://bucket.s3.amazonaws.com/key`
  - Supports **time-limited authenticated URLs**.
  - Can log HTTP access data for later analysis.
  - Designed so developers focus on **innovation**, not storage management.
- 

### Early S3 Use Case

- **NASA's Stardust@Home project** uses S3 to store and deliver **60 million images**.
  - Enabled rapid scaling without building custom infrastructure.
  - Ideal for short-term, high-volume data needs.
- 

### Nirvanix Cloud Storage

- Uses custom software on Intel storage servers across multiple U.S. locations.
- Built a **Storage Delivery Network (SDN)** using the **Internet Media File System (IMFS)**.
- Stores and delivers data from the **best geographic location** for performance.
- Supports multiple file copies across regions for high availability.

### CloudNAS

- Mounts cloud storage as a **virtual drive** using NFS, CIFS, or FTP.
- Integrates with existing backup and archive systems.
- Supports encryption, policy-based replication, and a single global namespace.
- Designed for backup, archiving, and unstructured data.

### Benefits

- **80–90% cost savings**
  - No large capital expenses
  - Instant data access
  - Built-in disaster recovery
  - Automated replication across up to three locations
  - Targeted at enterprises with long-term storage needs
- 

### Google Bigtable Datastore

- Introduced in 2008 to support **highly scalable databases**.
- Can scale to **petabytes** across **thousands of servers**.
- Available through **Google App Engine**.

### How Bigtable Works

- Uses **sparse multidimensional tables** with timestamps.
- Stores multiple versions of data.
- Tables split into **tablets (~200MB each)** distributed across servers.
- Enables load balancing and failure tolerance.
- Uses compression to free storage.
- Uses a **three-tier lookup system** (META0 → META1 → tablets).

### Issues & Costs

- Proprietary system → **vendor lock-in**.
  - Comparable pricing to Amazon S3:
    - CPU: ~\$0.10–0.12 per core-hour
    - Storage: ~\$0.15–0.18 per GB-month
    - Bandwidth in/out charged separately
- 

### Apple MobileMe

- Cloud service for **push email, contacts, and calendars**.
- Syncs across iPhone, iPod touch, Mac, and PC.
- Includes Mail, Contacts, Calendar, Gallery, and **iDisk**.
- Provides **20 GB storage**.
- Supports drag-and-drop, sharing, and instant synchronization.

- Subscription-based pricing with individual and family plans.
- 

### Microsoft Live Mesh

- A **software-plus-services** platform connecting devices via the cloud.
  - Enables file sharing, synchronization, and remote access.
  - Components:
    - Cloud service
    - Client software (MOE)
    - Developer platform
  - Supports Windows, Mac, Windows Mobile.
  - Includes **Live Desktop** (online storage) and **Remote Desktop**.
  - **Live Framework APIs** allow developers to sync any data type.
  - Uses **Mesh Objects** tracked for synchronization.
- 

### Key Takeaway

- Cloud storage provides **scalable, distributed, Internet-based storage**.
- It reduces infrastructure complexity but introduces concerns like **vendor lock-in, outages, and security**.
- Organizations should view cloud storage as **scalable IT capabilities delivered over the Web**, not just remote disks.

### Cloud Applications

- A cloud application eliminates the need to install and run software on the client computer.
  - Applications run on the cloud, so a **standard communication method** between client and cloud is required.
- 

### HTTP (Hypertext Transfer Protocol)

- HTTP is the **primary protocol** used to transfer data between clients and the cloud.
- It is a **stateless protocol**, meaning servers do not remember user information between requests.
- To maintain user state (e.g., login sessions), web applications use **cookies**.

- HTTP is the common “language” browsers and cloud services use to communicate.

### How HTTP Works

- A browser sends a request (e.g., GET) to a server.
  - The server responds with a status code (e.g., 200 OK) and the requested content.
  - HTTP 1.1 improves efficiency by allowing **persistent connections**, avoiding repeated connection setup for page elements.
  - HTTP defines **eight request methods** describing how actions are performed on server data.
  - HTTP is currently the **most common cloud access protocol**.
- 

### XMPP (Extensible Messaging and Presence Protocol)

- XMPP is considered a potential future protocol for cloud computing.
- Unlike HTTP-based protocols (SOAP, REST), which are **one-way**, XMPP supports **two-way, real-time communication**.
- Eliminates the need for **polling** (repeated client requests to check for updates).

### Polling Problem

- Polling increases load and inefficiency (e.g., email checking, web APIs).
- High polling rates can overwhelm servers, as seen with Twitter outages in 2008.
- Some systems attempt callbacks to avoid polling, but this complicates firewall configuration.

### Why XMPP Isn't Widely Used Yet

- It is not based on HTTP, which limits adoption.
  - Originally designed for instant messaging.
  - Advantages:
    - Two-way communication
    - XML-based and extensible
    - Scales to millions of users
  - Expected to gain importance as cloud needs evolve.
- 

### Security in Cloud Communication

- Cloud sessions are secured using **encryption** and **authentication**.
  - The most common encryption method is **SSL**.
- 

### Secure Sockets Layer (SSL)

- SSL creates an **encrypted connection** between a browser and a cloud server.
- Requires an **SSL certificate** issued by a trusted Certificate Authority (CA).
- The certificate uses **public and private cryptographic keys**.
- The public key is included in a **Certificate Signing Request (CSR)**.
- Once verified, the CA issues the certificate, enabling secure communication.

#### SSL Connection Process

- Browser verifies the site's certificate.
- Browser and server agree on encryption type.
- Encryption keys are exchanged.
- Secure communication begins.
- Browser displays a security indicator (lock icon).

#### SSL Certificate Details

- Contains domain name, organization name, address, country, expiration date, and issuing authority.
  - Browser checks:
    - Certificate validity
    - Expiration date
    - Trusted issuing authority
  - If checks fail, the browser warns the user.
- 

#### Key Takeaway

- **HTTP** is the backbone of cloud communication.
- **XMPP** offers real-time advantages but is not yet widely adopted.
- **SSL** ensures secure, private communication between users and cloud services.

#### OpenID

- OpenID is an **open-source authentication system** that lets users log in to multiple websites using **one identity**.
- Users can choose an OpenID provider they **trust** and keep the same identity even if they change providers.
- OpenID is **free**, reducing costs for businesses related to password and account management.

- It is a **lightweight authentication mechanism** based on web identification technologies.
  - Adoption is growing, with major companies like **AOL, Microsoft, Sun, and Novell** supporting it.
  - Anyone can be an OpenID **user or provider** at no cost.
- 

### PCI DSS (Payment Card Industry Data Security Standards)

- PCI DSS requirement **2.2.1** states that only one primary function should run per server.
  - This does **not** mean one physical server; **virtualized systems** are allowed.
  - Virtual machines must be **segmented and isolated** from each other.
  - PCI DSS rules are expected to **evolve** as technologies like virtualization change computing.
  - Similar to copyright laws adapting to new technologies, PCI DSS will continue to change.
- 

### How HTML Works

- HTML consists of **tags** written in a text file and interpreted by a web browser.
- Browsers like Internet Explorer or Firefox translate HTML into visual web pages.
- HTML can be written using **simple text editors** or advanced graphical tools.

### HTML Tags

- Tags are enclosed in **angle brackets (< >)** and control how content appears.
  - Tags define elements like **text formatting, images, and tables**.
  - Tags do not appear on the page but affect browser behavior (e.g., <b> makes text bold).
- 

### Cascading Style Sheets (CSS)

- CSS controls the **presentation and layout** of web pages.
  - It improves accessibility and separates content from design.
  - **JavaScript** adds interaction and dynamic behavior to HTML and CSS.
- 

### Dynamic HTML (DHTML)

- DHTML is not a new language, but a **technique** to make web pages dynamic.
- Allows page elements to change **without reloading** the page.
- Regular HTML pages remain static until a new server request is made.

### Components of DHTML

- Document Object Model (DOM)
  - Scripts
  - Cascading Style Sheets (CSS)
  - XHTML / HTML 4.x
- 

### Document Object Model (DOM)

- DOM represents every part of a web page as an object.
  - Provides consistent naming so scripts can **access and modify page elements** dynamically.
- 

### Scripts

- Scripts control DOM elements.
  - Common scripting languages include **JavaScript** and **ActiveX**.
- 

### CSS in DHTML

- CSS controls colors, fonts, backgrounds, and layout.
  - Scripts and DOM allow styles to be **changed dynamically**.
- 

### XHTML

- DHTML pages are written using **XHTML or HTML 4.x**.
  - XHTML provides structure for CSS and DOM manipulation.
- 

### Key Features of DHTML

- Changing HTML tags and properties dynamically
  - Real-time positioning of elements
  - Dynamic fonts (primarily Netscape-based)
  - Data binding (Internet Explorer-based feature)
- 

### JavaScript

- JavaScript adds **logic and interactivity** to web pages.
- Enables features like:
  - Opening and controlling browser windows

- Form validation
  - Image rollovers
  - JavaScript runs on the **client-side**, making it fast and responsive.
  - It detects user actions like keystrokes, which HTML cannot.
  - Though named similarly, JavaScript is **different from Java**.
  - Influenced by Java syntax but designed to be simpler.
  - Used extensively in modern frameworks like **React, Angular, Vue.js, Node.js, and jQuery**.
- 

### JavaScript Execution

- Browsers expose DOM elements as **host objects** accessible via JavaScript.
- JavaScript can also be used on servers to manipulate HTTP requests and responses.
- If JavaScript is disabled, browsers fall back to <noscript> content.

### Infrastructure

- Infrastructure delivers **virtualization** in cloud computing.
  - Virtualization can be implemented **locally or over the Internet** to support cloud services.
- 

### Virtualization

- Virtualization standards were developed collaboratively by major companies including **VMware, Intel, AMD, IBM, HP, Dell, Cisco, Red Hat**, and others.
- VMware introduced **VMware Community Source**, allowing partners access to **ESX Server source code and APIs**.
- The goal is **open standards**, shared governance, and collaborative development.
- Open virtualization enables customers to fully leverage the **x86 virtualization layer**.

### Benefits to End Users

- Faster availability of new virtualization solutions
  - Better interoperability across different vendors
  - Support for heterogeneous virtualized environments
  - Faster development of virtualization-aware technologies
-

## Open Hypervisor Standards

- Hypervisors enable **system partitioning** and are the foundation of virtual infrastructure.
  - Open hypervisor standards encourage **innovation and interoperability**.
  - VMware contributed **VMHI (Virtual Machine Hypervisor Interfaces)**.
  - Focus areas include:
    - Standardized management of virtual machines
    - APIs between hypervisors and guest OS
    - VM formats for migration and recovery across platforms
- 

## Community Source

- VMware Community Source provides **royalty-free access** to ESX Server source code.
  - Partners can contribute shared code or binary modules.
  - Combines benefits of **open-source and commercial models**.
  - Partners influence governance via an architecture board.
  - Customers benefit from **broader, well-integrated solutions**.
  - Partners can build **differentiated, IP-protected products**.
- 

## OVF (Open Virtualization Format)

- OVF is a **vendor-neutral standard** for packaging virtual appliances.
- Supports one or more virtual machines.
- Prevents **vendor lock-in** and allows choice of hypervisors.
- Accelerates adoption of virtual appliances.

## OVF Features

- Portable and platform-independent
- Supports integrity checks and licensing
- Automated and simple installation
- Validates compatibility with local hardware
- Supports all current and future virtual disk formats
- Accurately captures VM properties

## OVF Characteristics

- Vendor and platform independent

- Extensible for future technologies
  - Localizable for multiple regions and languages
- 

## Service

- A **web service** enables machine-to-machine interaction over a network.
  - Often implemented as **web APIs** accessed over the Internet.
- 

## Data

- Cloud data is commonly exchanged using **JSON** and **XML**.
  - Both are based on industry standards and support web applications.
- 

## JSON

- JSON (JavaScript Object Notation) is a **lightweight data format**.
- Used for **data serialization** and transmission.
- Language-independent with support across many platforms.
- Ideal for **AJAX and JavaScript-based applications**.

## JSON vs XML

- JSON is preferred when using JavaScript.
  - JSON objects can be used directly without DOM parsing.
  - Reduces CPU usage, bandwidth, and code complexity.
- 

## XML

- XML is a **self-describing, platform-independent data format**.
- Separates **data structure from presentation**.
- Extensible: developers define their own tags.
- Supports structured, hierarchical data storage.

## Why XML is Important

- Handles rich data (audio, video, nested data)
- Ideal for cloud and distributed environments
- XML databases store data natively for faster queries
- Eliminates complex mapping to relational databases

## Benefits of XML

- Self-describing and machine-readable
  - Supports multimedia and complex data
  - Easy updates using XSL without reprogramming
  - Appears as a **single logical database** in the cloud
  - Open, extensible, and future-proof (W3C standard)
- 

## Web Services

- Web services define how cloud data reaches clients.
  - Two main approaches: **REST** and **SOAP**.
- 

## REST (Representational State Transfer)

- Uses **HTTP and XML/JSON** to access resources via URLs.
- Simple, lightweight, and easy to use.
- Developed by **Roy Fielding** as an architectural style.
- Ideal for reading and publishing cloud data.
- Uses the same model as **RSS** feeds.

## Benefits of REST

- Faster response and lower server load
  - Better scalability (stateless)
  - Works with standard browsers
  - No separate discovery mechanism needed
  - Easy bookmarking and sharing of queries
  - Forward and backward compatibility
- 

## SOAP (Simple Object Access Protocol)

- Enables programs on different operating systems to communicate.
- Uses **HTTP and XML**, making it firewall-friendly.
- Defines strict rules for encoding requests and responses.
- More complex but more powerful than REST.
- Suitable for **enterprise-level integrations**.

## SOAP Advantages

- Works across firewalls and proxies
  - Standardized communication format
  - Supports complex transactions and operations
- 

## Key Takeaways

- Virtualization enables scalable cloud infrastructure.
- Open standards improve interoperability and innovation.
- OVF prevents vendor lock-in.
- JSON is lightweight and ideal for web apps.
- XML is powerful for structured, complex data.
- REST is simple and scalable; SOAP is robust and feature-rich.

## Web Browsers

- Web browsers are the primary tools used by users to access cloud services.
  - Browser choice affects compatibility, performance, and user experience.
  - Market share (Dec 2008):
    - Internet Explorer: **69.77%**
    - Mozilla Firefox: **20.78%**
    - Apple Safari: **7.13%**
    - Google Chrome: **0.98%**
    - Others (Opera, Camino, etc.): **~2%**
  - Internet Explorer's dominance is mainly due to its bundling with Windows.
  - Market shares change slowly but remain relatively stable over time.
- 

## Internet Explorer

- Internet Explorer 8 runs on Windows Vista, XP, and Windows 7.

## IE 8 Features

- Improved user interface and faster browsing, searching, and printing.

- Introduced **multiple rendering modes** to balance modern standards and backward compatibility.
  - Designers can specify rendering mode; otherwise, a default mode is used.
  - Three rendering modes:
    - Current web standards mode
    - IE7 standards mode (2006 behavior)
    - Legacy mode for early web content
  - Forward-looking mode is preferred, older modes ensure compatibility.
  - Designed for developers and IT administrators.
  - Strong enterprise support via **Active Directory Group Policy**.
  - Enables centralized deployment and management across networks.
- 

## Firefox

- Firefox 3 released in June 2008 as a major open-source update.
- Developed by a global community over three years.
- Available in around **50 languages**.
- 2–3× faster than Firefox 2 with over **15,000 improvements**.

## User Experience

- Smart Location Bar (“Awesome Bar”) adapts to user behavior.
- Library manages history, bookmarks, and tags.
- One-click bookmarking and tagging.
- Full-page zoom improves readability.

## Firefox Performance

- Built on **Gecko 1.9** platform.
- Uses less memory and loads pages faster than previous versions.

## Security

- Malware and phishing protection.
- One-click site identity verification.
- Benefits from Mozilla’s global open-source security community.

## Customization

- Supports over **5,000 add-ons**.

- Add-ons allow extended functionality directly in the browser.
  - Improved Add-ons Manager for easy installation.
- 

## Safari

- Safari 3.1 available for Mac OS X and Windows.
- Claims faster page loads than IE 7 and Firefox 2.
- JavaScript execution up to **6× faster**.
- First browser to support advanced Web 2.0 standards.

## Safari Performance

- Intuitive interface with drag-and-drop bookmarks and tab management.
- Integrated search and built-in RSS reader.

## Standards Support

- First to support **HTML5 audio/video tags**.
- Supports **CSS Animations** and **CSS Web Fonts**.

## System Requirements

- Mac: OS X Tiger 10.4.11 or Leopard, 256MB RAM.
  - Windows: XP or Vista, 256MB RAM, 500MHz Pentium processor.
- 

## Chrome

- Google Chrome is Google's open-source browser.
- Designed for modern, application-heavy web usage.
- Focuses on speed, simplicity, and stability.
- Simple interface with a powerful underlying architecture.

## Chrome Features

- Combined address and search bar.
  - New tab page shows frequently visited sites and bookmarks.
  - Each tab runs as a separate process, improving stability.
  - Uses **V8 JavaScript engine** for high-speed execution.
- 

## Open Source

- Chrome is based on open-source projects.

- Released as **Chromium** to encourage innovation.
  - Google contributes browser technologies to the wider community.
- 

### Chrome Cloud

- Chrome strongly supports cloud computing.
- Powered by the V8 engine and **Google Gears**.

### Google Gears Features

- Local database storage
  - Parallel JavaScript execution (WorkerPool)
  - Offline caching of application resources
  - Desktop integration
  - Geolocation support
- 

### Final Notes

- Chrome enables convergence of desktop and web applications.
- Acts as a virtual machine for online and offline web apps.
- Available at **google.com/chrome**.
- Chromium available at **chromium.org**.
- Browser choice depends on:
  - User skills
  - Platform compatibility
  - Developer requirements
  - Vendor support